

Lab 3: Linear and Circular Queues

Module	ET4047
Date	27/09/2026
Lab No.	Lab 3
Author	Luke Griffin

1 Overview

A **queue** is a First-In, First-Out (FIFO) data structure: elements join at the **rear** and leave from the **front**, just like people in a queue at a shop. In this lab you will run two array-based queue implementations on an Arduino Nano in the Wokwi simulator, work through how each function operates, and then fix a real flaw in the linear version.

The flaw: in the linear queue, every `dequeue()` permanently uses up a slot at the start of the array. Once `rear` reaches the last slot, the queue refuses new elements even when most of the array is empty. By the end of this lab your queue will reuse freed space, so it can be filled and emptied indefinitely.

Learning outcomes

By the end of this lab you should be able to:

- Explain how `front`, `rear` and `count` describe the state of an array-based queue.
- Trace `enqueue()`, `dequeue()`, `peek()` and `enqueueToOppositeEnd()` by hand and on hardware.
- Demonstrate why a linear queue wastes space after `dequeue()`.
- Fix the problem two ways: by shifting elements, and by circular (modulo) indexing.
- Compare the time cost of each approach using Big-O notation.

What you need

- A web browser and the Wokwi simulator: wokwi.com → *New Project* → **Arduino Nano**.
- The two example sketches: `Linear_Queue_Example.ino` and `Circular_Queue_Example.ino`.
- The lecture notes *W04 Linear Queue vs Circular Queue* and the *Queue Data Structure Tutorial*.

Project set-up

1. Create a new Arduino Nano project in Wokwi and name it `Linear_Queue_Example`.
2. Replace the contents of `sketch.ino` with the linear queue example code.
3. Open `diagram.json` and check it matches the listing below. The `serialMonitor` entry makes the Serial Monitor open automatically.

- Press the green **Play** button and confirm the output matches the expected output in Section 3.7.
- Repeat steps 1 to 4 for a second project called `Circular_Queue_Example` using the circular code.

diagram.json

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [ { "type": "wokwi-arduino-nano", "id": "nano", "top": 0, "left": 0, "attrs": {} } ],
  "connections": [],
  "serialMonitor": { "display": "always", "newline": "\n" },
  "dependencies": {}
}
```

No wiring needed

Everything in this lab happens in software. The Nano is only there to run the code; all results appear in the Serial Monitor at 9600 baud.

2 Queue operations recap

Both example sketches implement the same set of operations. Only the way they move `front` and `rear` through the array differs.

Operation	Function in the code	What it does
Create	<code>initializeQueue(&q)</code>	Sets up an empty queue.
Enqueue	<code>enqueue(&q, x)</code>	Adds <code>x</code> at the rear . Returns <code>false</code> if the queue is full.
Enqueue on opposite end	<code>enqueueToOppositeEnd(&q, x)</code>	Adds <code>x</code> at the front , so it jumps the queue.
Dequeue	<code>dequeue(&q)</code>	Removes and returns the front element. Returns <code>-1</code> if empty.
Peek	<code>peek(&q)</code>	Returns the front element without removing it.
Is empty / Is full	<code>isEmpty(&q) / isFull(&q)</code>	Report whether the queue can be dequeued from / enqueued to.
Destroy	<code>destroyQueue(&q)</code>	Resets the queue to empty.

Every function takes a **pointer** to the queue (`Queue *q`) so it can modify the caller's queue directly. Inside the function, `q->front` means "the `front` field of the queue that `q` points to". In `setup()` we pass the address of our queue with `&myQueue`.

3 Walkthrough: the linear queue

Open `Linear_Queue_Example` in Wokwi and follow along. Each section shows a piece of the code and then explains it line by line.

3.1 The Queue structure

```
#define MAX_QUEUE_SIZE 10

typedef struct
{
    int items[MAX_QUEUE_SIZE];
    int front;
    int rear;
    int count;
} Queue;
```

`#define MAX_QUEUE_SIZE 10` fixes the capacity at compile time. The Nano has only 2 KB of RAM, so fixed-size arrays are the normal approach on microcontrollers. The `typedef struct` bundles four pieces of data into a single type called `Queue`:

Field	Meaning
<code>items[]</code>	The array that stores the elements. Slots outside the queue can hold old leftover values, and that is fine.
<code>front</code>	Index of the element that will be removed next.
<code>rear</code>	Index of the element that was added most recently. <code>-1</code> means nothing has been added yet.
<code>count</code>	How many elements are currently in the queue.

Key idea

The array never physically “moves”. The queue is simply the slots from `front` to `rear`. Enqueue and dequeue work by moving these two index markers.

3.2 initializeQueue(), isEmpty() and isFull()

```
void initializeQueue(Queue *q)
{
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}

bool isEmpty(Queue *q)
{
    return q->count == 0;
}

bool isFull(Queue *q)
{
    return q->rear == MAX_QUEUE_SIZE - 1;
}
```

- `initializeQueue()` puts the queue into its empty state: `front = 0`, `rear = -1`, `count = 0`. Setting `rear` to `-1` means the first `enqueue()` will increment it to `0`, the first slot.
- `isEmpty()` uses `count`. If no elements are stored, the queue is empty. Simple and reliable.

- `isFull()` is different. It does **not** look at `count`; it checks whether `rear` has reached the **last index** of the array. Keep an eye on this line. It is the root of the problem you will fix later.

3.3 enqueue(): add to the rear

```
bool enqueue(Queue *q, int value)
{
    if (isFull(q))
    {
        Serial.println("Error: Queue is full. Cannot enqueue.");
        return false;
    }

    q->rear++;
    q->items[q->rear] = value;
    q->count++;

    Serial.print("Enqueued: ");
    Serial.println(value);

    return true;
}
```

1. **Guard clause:** if the queue is full, print an error and return `false` so the caller knows the value was not stored.
2. `q->rear++` moves the rear marker one slot to the right.
3. `q->items[q->rear] = value` stores the value in that new slot.
4. `q->count++` records that there is one more element.
5. Print a confirmation and return `true`.

Every step is a single operation, so `enqueue()` runs in constant time, **$O(1)$** .

After enqueue 10, 20, 30 (5-slot example)

index	0	1	2	3	4
items[]	10	20	30	-	-
	▲ front		▲ rear		

3.4 enqueueToOppositeEnd(): add to the front

```
bool enqueueToOppositeEnd(Queue *q, int value)
{
    if (isFull(q))
    {
        Serial.println(
            "Error: Queue is full. Cannot enqueueToOppositeEnd."
        );
        return false;
    }

    // If the queue is empty
    if (isEmpty(q))
    {
        q->front = 0;
        q->rear = 0;
        q->items[q->front] = value;
    }
    else
    {
        // Shift all elements one position to the right
        for (int i = q->rear; i >= q->front; i--)
        {
            q->items[i + 1] = q->items[i];
        }

        // Insert the new element at the front
        q->items[q->front] = value;

        // Rear moves one position to the right
        q->rear++;
    }

    q->count++;

    Serial.print("Enqueued to opposite end: ");
    Serial.println(value);

    return true;
}
```

This lets an element jump the queue: it becomes the next one to be removed. In a linear array there is no free slot *before* the front element (when **front** is 0), so the function has to make room:

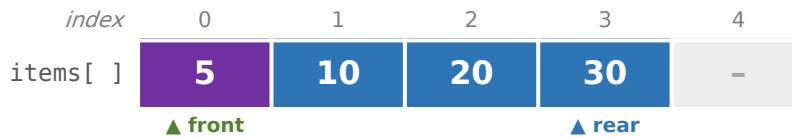
- **Empty queue:** just place the value at index 0 and set both **front** and **rear** to 0.
- **Otherwise:** the **for** loop walks **backwards** from **rear** to **front**, copying each element one place to the right. It must go backwards; going forwards would overwrite elements before they are copied.
- The new value is written into the now-vacant **items[front]**, and **rear** moves right by one.

Because it may move every element, this operation is **O(n)**.

Before: enqueueToOppositeEnd(&q, 5)

index	0	1	2	3	4
items[]	10	20	30	-	-
	▲ front		▲ rear		

After: 10, 20 and 30 shifted right; 5 written at the front



3.5 dequeue(): remove from the front

```
int dequeue(Queue *q)
{
    if (isEmpty(q))
    {
        Serial.println(
            "Error: Queue is empty. Cannot dequeue."
        );
        return -1;
    }

    int item = q->items[q->front];

    q->front++;
    q->count--;

    // Reset queue when the last element is removed
    if (q->count == 0)
    {
        q->front = 0;
        q->rear = -1;
    }

    Serial.print("Dequeued: ");
    Serial.println(item);

    return item;
}
```

1. **Guard clause:** you cannot remove from an empty queue, so print an error and return `-1`. (This means `-1` itself cannot be stored safely. Think about how you might fix that.)
2. Save the front element in `item` *before* moving anything.
3. `q->front++` moves the front marker right. The old slot is **not erased**; it simply stops being part of the queue.
4. `q->count--` records one fewer element.
5. If the queue is now empty, reset `front` and `rear` to their starting values so the whole array is available again.

Notice

The reset in step 5 only happens when the queue becomes **completely** empty. If even one element remains, the slots before `front` stay unusable. Section 4 shows why this matters.

3.6 peek() and destroyQueue()

```
int peek(Queue *q)
{
    if (isEmpty(q))
    {
        Serial.println(
            "Error: Queue is empty. Cannot peek."
        );

        return -1;
    }

    return q->items[q->front];
}

void destroyQueue(Queue *q)
{
    q->front = 0;
    q->rear = -1;
    q->count = 0;

    Serial.println("Queue destroyed.");
}
```

- `peek()` returns `items[front]` but changes nothing, so it is useful when you want to look at the next element before deciding whether to remove it.
- `destroyQueue()` sets the fields back to the empty state. Because the queue is defined only by `front`, `rear` and `count`, there is no need to clear the array. The old values are simply ignored.

3.7 setup(), loop() and expected output

`setup()` runs once at power-up. It initialises the queue, enqueues 10, 20 and 30, pushes 5 onto the front, peeks, and finally destroys the queue. `loop()` is empty because nothing needs to repeat. You should see:

Serial Monitor output

```
Queue initialized.

Enqueued: 10
Enqueued: 20
Enqueued: 30

Enqueued to opposite end: 5

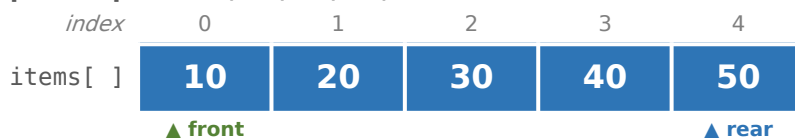
Front element: 5

Queue destroyed.
```

4 The problem: dequeued space is lost

Suppose `MAX_QUEUE_SIZE` is 5 and we enqueue five elements. `rear` is now 4, the last index:

Step 1: enqueue 10, 20, 30, 40, 50



Now dequeue twice. `front` moves to 2 and `count` drops to 3. Slots 0 and 1 are free:

Step 2: dequeue twice



Try to enqueue 60. `isFull()` checks `rear == MAX_QUEUE_SIZE - 1`, which is `4 == 4`, so it returns `true`. The queue reports “Queue is full” even though only 3 of 5 slots are in use:

Step 3: enqueue(60) is rejected



Slots 0 and 1 are empty but can never be reached, because `rear` can only move right.

Why this matters on a microcontroller

Queues are used as buffers: serial bytes arriving, button events, sensor samples waiting to be processed. Items are added and removed continuously, and the queue rarely becomes completely empty. With the linear design, the buffer “fills up” after a fixed number of total insertions and then silently drops data. That bug is very hard to spot in a real device.

There are two standard fixes:

- **Shift on dequeue:** after removing the front element, slide everything left so `front` is always 0. Simple, but every dequeue costs $O(n)$.
- **Circular indexing:** let `front` and `rear` wrap around from the last slot back to slot 0. Every operation stays $O(1)$. This is exactly what `Circular_Queue_Example` does.

5 Walkthrough: the circular queue

Open `Circular_Queue_Example`. The struct, `initializeQueue()`, `isEmpty()`, `peek()` and `destroyQueue()` are **identical** to the linear version. Only four functions change:

Function	Linear queue	Circular queue
isFull()	<code>rear == MAX_QUEUE_SIZE - 1</code>	<code>count == MAX_QUEUE_SIZE</code>
enqueue()	<code>rear++</code>	<code>rear = (rear + 1) % MAX_QUEUE_SIZE</code>
dequeue()	<code>front++</code> (plus reset when empty)	<code>front = (front + 1) % MAX_QUEUE_SIZE</code>
enqueueToOppositeEnd()	Shift all elements right: O(n)	<code>front = (front - 1 + MAX_QUEUE_SIZE) % MAX_QUEUE_SIZE;</code> O(1)

5.1 How the modulo operator makes the array “wrap”

`%` gives the remainder after division. Adding 1 and taking `% MAX_QUEUE_SIZE` counts upwards normally until it hits the end of the array, then jumps back to 0. For a 5-slot queue:

rear before	rear + 1	(rear + 1) % 5	rear after
0	1	1	1
1	2	2	2
2	3	3	3
3	4	4	4
4	5	0	0 ← wraps to the start

Circular enqueue()

```
bool enqueue(Queue *q, int value)
{
    if (isFull(q))
    {
        Serial.println("Error: Queue is full. Cannot enqueue.");
        return false;
    }

    // Move rear circularly
    q->rear = (q->rear + 1) % MAX_QUEUE_SIZE;

    q->items[q->rear] = value;
    q->count++;

    Serial.print("Enqueued: ");
    Serial.println(value);

    return true;
}
```

Going **backwards** needs one extra step. In C, `(0 - 1) % 5` gives `-1`, which is not a valid index. Adding `MAX_QUEUE_SIZE` first keeps the number positive: `(0 - 1 + 5) % 5 = 4`. So stepping back from slot 0 lands on the last slot:

Circular enqueueToOppositeEnd()

```
bool enqueueToOppositeEnd(Queue *q, int value)
{
    if (isFull(q))
    {
        Serial.println(
            "Error: Queue is full. Cannot enqueueToOppositeEnd."
        );
        return false;
    }

    // Move front one position backwards circularly
    q->front = (q->front - 1 + MAX_QUEUE_SIZE)
               % MAX_QUEUE_SIZE;

    q->items[q->front] = value;
    q->count++;

    Serial.print("Enqueued to opposite end: ");
    Serial.println(value);

    return true;
}
```

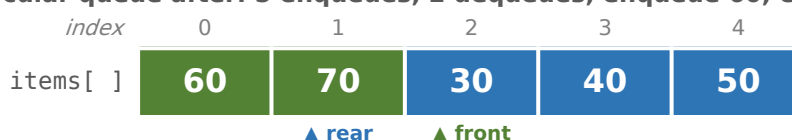
No shifting loop is needed at all: the new element goes into the slot just “behind” the front, wrapping round to the end of the array if necessary.

5.2 Why isFull() must use count

Once the markers wrap, **rear** can be *less* than **front**, so comparing **rear** to the last index no longer tells you anything. **count** is the one value that always tells you how many elements are stored, so the queue is full exactly when **count == MAX_QUEUE_SIZE**.

Replaying the scenario from Section 4 with the circular queue, 60 and 70 now fill the freed slots 0 and 1:

Circular queue after: 5 enqueues, 2 dequeues, enqueue 60, enqueue 70



Reading order starts at **front** (index 2): 30, 40, 50, then wraps to 60, 70. **rear** (1) is now *behind* **front** in the array.

6 Lab tasks

Work in your **Linear_Queue_Example** project unless a task says otherwise. Save a copy of the sketch after each task (for example **Task3.ino**, **Task4.ino**) so you can go back to it. Paste the Serial Monitor output into your lab report for every task marked **(record)**.

Task 1: Run and compare both examples

1. Run both example sketches in Wokwi. **(record)** Paste both outputs into your report.
2. The two outputs are identical. Explain in one or two sentences why this test cannot reveal any difference between the two designs.

Your answer:

Task 2: Add a `printQueue()` function

Debugging a queue is much easier when you can see inside it. Add a function to the linear sketch with this signature, placed **above** `setup()`:

```
void printQueue(Queue *q);
```

It should print `front`, `rear` and `count`, then every slot of the array. Show the value for slots that are **in** the queue and `[--]` for slots that are not. For example:

Required format

```
front=2 rear=4 count=3 | [--] [--] [30] [40] [50]
```

Hint

In the linear queue, slot `i` is part of the queue when `count > 0` **and** `front <= i <= rear`. Use a `for` loop over all `MAX_QUEUE_SIZE` slots.

Task 3: Expose the problem

1. Change `#define MAX_QUEUE_SIZE 10` to `5`.
2. Replace the body of `setup()` (keep `Serial.begin(9600);`) with the test sequence below.
3. Run it. **(record)** Paste the output into your report.

Task 3 test sequence

```
initializeQueue(&myQueue);  
  
enqueue(&myQueue, 10);  
enqueue(&myQueue, 20);  
enqueue(&myQueue, 30);  
enqueue(&myQueue, 40);  
enqueue(&myQueue, 50);  
printQueue(&myQueue);  
Serial.println();  
  
dequeue(&myQueue);  
dequeue(&myQueue);  
printQueue(&myQueue);  
Serial.println();  
  
enqueue(&myQueue, 60);  
enqueue(&myQueue, 70);  
printQueue(&myQueue);  
Serial.println();
```

Answer: (a) How many elements are in the queue when 60 is rejected? (b) Which line of code causes the rejection? (c) Would the problem go away if you made `MAX_QUEUE_SIZE` bigger? Explain.

Your answer:

Task 4: Fix 1, shift elements on dequeue

Modify `dequeue()` so that after the front element is saved, every remaining element slides **one place to the left**. `front` then always stays at 0, and the freed slot moves to the end of the array where `enqueue()` can reuse it.

- Write a `for` loop that copies `items[i + 1]` into `items[i]` for every element in the queue. Should it run forwards or backwards this time? Why?
- Remove the `q->front++` line and decrement `rear` instead.
- Change `isFull()` to `return q->count == MAX_QUEUE_SIZE;`. This is the clearest definition of “full”.

Re-run the Task 3 test. **(record)** 60 and 70 should now be accepted. Your final `printQueue()` line should show:

Expected final line

```
front=0 rear=4 count=5 | [30] [40] [50] [60] [70]
```

Think about it

This works, but what does it cost? If the queue holds n elements, how many copy operations does one `dequeue()` now perform? Record your answer in the Section 7 table.

Task 5: Fix 2, convert the linear queue to a circular queue

Start from your **Task 3** version (not Task 4) and turn it into a circular queue yourself.

Do not copy the circular example; use it only to check your work afterwards.

1. `isFull()`: use `count` instead of `rear`.
2. `enqueue()`: replace `rear++` with a wrap-around increment using `%`.
3. `dequeue()`: replace `front++` with a wrap-around increment. Is the “reset when empty” block still needed? Try it with and without.
4. `enqueueToOppositeEnd()`: delete the shifting loop entirely. Step `front` backwards with wrap-around and write the value there. If the queue was empty beforehand, make `rear` equal `front` too.
5. `printQueue()`: your Task 2 test `front <= i <= rear` no longer works once `rear` wraps behind `front`. Fix it using the hint below.

Hint for `printQueue()`

Work out how far slot `i` is from the front, moving forwards with wrap-around:

```
int offset = (i - q->front + MAX_QUEUE_SIZE) % MAX_QUEUE_SIZE;
```

Slot `i` is part of the queue when `offset < q->count`.

Re-run the Task 3 test. **(record)** The final line should show 60 and 70 stored in slots 0 and 1:

Expected final line

```
front=2 rear=1 count=5 | [60] [70] [30] [40] [50]
```

Task 6: Stress test

A single test is not proof. Add the function below to your sketch and call it at the end of `setup()` (after `destroyQueue(&myQueue);`). It keeps one element in the queue at all times, so the “reset when empty” trick never rescues it, and then pushes 20 more values through a 5-slot queue, checking they come out in the right order.

Task 6 stress test

```
void stressTest(Queue *q)
{
    int next = 1;        // next value to enqueue
    int expected = 1;    // value we expect dequeue() to return
    int failures = 0;

    // Keep one item in the queue so it never becomes empty
    enqueue(q, next++);

    for (int round = 1; round <= 10; round++)
    {
        for (int i = 0; i < 2; i++)
        {
            if (!enqueue(q, next++)) failures++;
        }
        for (int i = 0; i < 2; i++)
        {
            if (dequeue(q) != expected++) failures++;
        }
    }

    Serial.print("Stress test finished. Failures: ");
    Serial.println(failures);
}
```

1. Run the stress test on the **original** linear queue (Task 3 version). **(record)** Record the number of failures.
2. Run it on your **Task 4** and **Task 5** versions. **(record)** Both must report **Failures: 0**.
3. Look at the original version's output. At which value does the first "Queue is full" error appear, and why at that point?

Your answer:

Task 7 (extension): Interactive queue over Serial

Make the queue interactive using the Wokwi Serial Monitor input box. In `loop()`, read a line with `Serial.readStringUntil('\n')` and respond to these commands, calling `printQueue()` after each one:

Command	Action
e 42	enqueue 42
f 7	enqueueToOppositeEnd 7
d	dequeue and print the value
p	peek and print the value
c	clear (destroyQueue)

Use it to demonstrate the wrap-around live: fill the queue, dequeue a few, then enqueue again. Hint: `String.toInt()` converts text to a number, and `Serial.available()` tells you when input has arrived.